ST5230 Assignment 1

Ng Jun Wei (A0252142U)
National University of Singapore
https://github.com/awpbash/ST5230_Language_Modelling

1 Overview

This report compares four families of language models (n-gram, RNN, LSTM, Transformer) trained on the AG News dataset, a balanced 4-class topic classification corpus (World, Sports, Business, Sci/Tech) with 96,000 training articles (∼4.7M tokens) and 7,600 test articles. Tokenization is whitespace-based with lowercasing, retaining the top 20,000 tokens. Sequences are truncated to 128 tokens. The training data is split 80/10/10 for language modeling; the original HuggingFace test set (1,900 per class) is used for downstream classification.

To ensure fair comparison, all neural models share the same dimensionality (embed_dim = hidden_dim = 100), depth (2 layers), dropout (0.1), optimizer (Adam, lr = 0.001), gradient clipping (5.0), batch size (64), and are trained for 20 epochs. Parameter counts range from 4.06M to 4.28M, so differences in performance reflect architecture rather than capacity. The shared embedding (20K × 100 = 2M) and output projection account for most parameters; differences come only from model cores.

All experiments use seed 42 with hyperparameters centralized in a single YAML config, and were run on one NVIDIA RTX 4060 GPU. Word2Vec embeddings (Part II) were trained only on the training split, and zero overlap between splits was verified. Before full training, we confirmed that all models can overfit a single batch (>50% loss drop), all parameters receive gradients, and the Transformer's causal mask prevents future token leakage.

2 Part I: Language Model Training and Comparison

To find out the performance, training time and inference time of different language model architectures, we train five language models on the same data: a bigram and trigram with Laplace smoothing (k=1), a 2-layer vanilla RNN (tanh), a 2-layer LSTM, and a 2-layer decoder-only Transformer (GPT-style, 4 heads, d_{ff}=400, learned positional embeddings, causal mask). All neural models are autoregressive and trained with cross-entropy on next-token prediction.

Model	**Architecture**	**Parameters**
Bigram	Count-based, Laplace smoothing	N/A
Trigram	Count-based, Laplace smoothing	N/A
RNN	2-layer vanilla RNN, tanh	4,060,400
LSTM	2-layer LSTM	4,181,600
Transformer	2-layer decoder-only, 4 heads	4,275,600

Table 1: Model architectures and parameter counts.

Model	Val PPL	Test PPL	Train Time	Inf. (ms/batch)
Bigram	860.44	849.76	5.3s	4.73
Trigram	3791.59	3735.70	10.0s	6.03
RNN	97.46	96.26	1878.5s	0.90
LSTM	78.75	77.82	1997.8s	0.94
Transformer	**54.07**	**53.59**	2101.9s	2.26

Table 2: Perplexity, training time, and inference speed.

Analysis

The Transformer achieves the lowest perplexity by a large margin, followed by LSTM then RNN (Table 2, Figure 1). This ordering makes sense: the RNN compresses all context into a fixed-size hidden state, which limits how far back it can "remember." The LSTM improves on this with gating that lets it carry information across longer spans. The Transformer performs best since self-attention lets every token look at every previous token directly, so there is no sequential bottleneck at all.

Interestingly, the trigram does *worse* than the bigram. This happens because Laplace smoothing spreads probability uniformly over unseen events, and the trigram has a much larger context space (V^2 possible bigram histories). Most trigram contexts are never seen in training, so the model ends up assigning near-uniform probabilities most of the time.

Training time increases modestly from RNN to Transformer (Table 2). The Transformer is also the slowest at inference due to $O(n^2)$ attention, though at this scale the difference is small. The train-val loss gap stays small for all models across 20 epochs (Figure 2), so none of them are significantly overfitting.

For text generation, we sample from each model using the prompt "the president of" (Table 3). The bigram immediately loses coherence after a couple of words since it only conditions on the previous token. The trigram produces longer fragments but still drifts rapidly. The RNN manages some topical relevance but mixes unrelated phrases. The LSTM holds slightly better but still wanders. The Transformer produces the most coherent output, generating a plausible news-like sentence about the Iraqi president that stays on topic throughout.

Model	Generated text (prompt: "the president of")
Bigram	the president of european union has been hard drives measuring 7.
Trigram	the president of haiti #39; technology stocks, after attackers blew up an independent inquiry into school siege remain in office before 2008 ...
RNN	the president of brazil #39; s rift after being contempt called stalking between the virginia empty runs thursday, killing 11 militants ...
LSTM	the president of back 3 - man - vs losing market chelsea, alex, who pulled the fight to a crucial career ...
Transformer	the president of iraqi aid reportedly diverted new iraqi president ghazi al - yawar arrived at a memorial center at a donors centre to former iraqi prime minister ayad allawi.

Table 3: Generated text samples (temperature=1.0, max 128 tokens, truncated for space).

3 Part II: Embedding Variants and Ablation

In this section, we explore the effect of different embeddings by replacing the default trainable embeddings with two frozen alternatives and retrain each neural model, giving a 3×3 grid of 9 experiments:

- **Scratch**: trainable 100-dim embeddings, randomly initialized.
- **W2V frozen**: Word2Vec (Skip-gram, dim=100, window=5) self-trained on the AG News training set only, then frozen.

- **GloVe frozen**: `glove-wiki-gigaword-100`, pretrained on Wikipedia + Gigaword from gensim package.

Freezing removes ∼2M embedding parameters from gradient updates, so frozen models have roughly half the trainable parameters.

Model	Params (scratch/frozen)	Scratch PPL	W2V PPL	GloVe PPL
RNN	4.06M / 2.06M	96.26	**93.13**	111.39
LSTM	4.18M / 2.18M	78.00	**75.06**	83.12
Transformer	4.28M / 2.28M	**53.83**	57.65	60.76

Table 4: Test perplexity by embedding variant. Bold = best per model.

Run	Epoch-1 PPL	Best Val PPL	Gap
RNN scratch	252.80	97.46	155.34
RNN W2V	242.86	94.18	148.68
RNN GloVe	239.55	112.84	126.71
LSTM scratch	358.39	79.01	279.38
LSTM W2V	464.46	75.96	388.51
LSTM GloVe	456.77	84.28	372.49
TF scratch	184.57	54.42	130.16
TF W2V	164.01	58.37	105.65
TF GloVe	157.18	61.39	95.79

Table 5: Epoch-1 perplexity and total improvement (convergence speed).

Analysis

For RNN and LSTM, Word2Vec (W2V) gives the best perplexity (Table 4). This makes sense because W2V was trained on the same news corpus, so it captures domain-specific patterns that the model can exploit immediately. GloVe, pretrained on a much larger but general-purpose corpus, is less aligned with news text and also has slightly lower vocabulary coverage (1.2% OOV tokens get zero vectors). Recurrent models process tokens through a fixed-size hidden state, so they benefit more from a good embedding initialization since they have limited capacity to "fix" bad input representations on the fly.

The Transformer on the other hand performed best with scratch embeddings (Table 4). This might be due to the self-attention mechanism which allows the Transformer to effectively learn its own optimal embeddings during training. Frozen embeddings actually hurt because they prevent the model from co-adapting the input space with the attention layers.

An interesting anomaly shows up in convergence speed (Table 5). For RNN and Transformer, pretrained embeddings give lower epoch-1 perplexity as expected, since they start from a better initialization. But for LSTM, scratch actually starts *lower* than the frozen variants. This might happen because LSTM gates are initialized assuming random-scale inputs; pretrained embeddings have a different magnitude distribution that causes the gates to saturate early on. Despite this slow start, W2V LSTM still converges to the best final perplexity (Figure 3).

GloVe is the worst embedding for every model suggesting that domain mismatch matters more than pretraining corpus size.

4 Part III: Downstream Task with Learned Representations

The main question here is whether better language models produce better features for classification. We extract frozen representations from each LM by running a forward pass and mean-pooling the final-layer hidden states across non-padding positions, producing one 100-dim vector per input. These features are then used for AG News 4-class topic classification.

We compare four approaches: (1) a bag-of-words baseline (CountVectorizer + logistic regression), (2) a frozen linear probe on LM features, (3) a frozen MLP probe (100→128→4, ReLU), and (4) end-to-end fine-tuning of the Transformer with a linear head using differential learning rates (backbone: 1e-4, head: 1e-3). All classifiers train for 10 epochs with Adam.

Method	**Description**
BoW + LogReg	CountVectorizer + logistic regression (sklearn)
Frozen LM + Linear	Frozen features → linear layer (100→4)
Frozen LM + MLP	Frozen features → MLP (100→128→4, ReLU)
Fine-tuned TF + Linear	End-to-end fine-tuning, differential LR

Table 6: Downstream classification methods.

Method	**Acc**	**F1**	**World**	**Sports**	**Business**	**Sci/Tech**
BoW + LogReg	0.902	0.902	0.906	0.961	0.865	0.877
RNN frozen + Linear	0.840	0.840	0.845	0.920	0.794	0.801
RNN frozen + MLP	0.857	0.857	0.867	0.930	0.812	0.818
LSTM frozen + Linear	0.871	0.871	0.873	0.945	0.824	0.844
LSTM frozen + MLP	0.883	0.883	0.882	0.948	0.842	0.859
TF frozen + Linear	0.880	0.879	0.886	0.947	0.839	0.846
TF frozen + MLP	0.906	0.906	0.915	0.963	0.869	0.876
TF fine-tuned + Linear	**0.931**	**0.931**	**0.939**	**0.978**	**0.901**	**0.904**

Table 7: Downstream classification results. Per-class columns are F1 scores.

Analysis

The most surprising result is that a simple BoW baseline beats every frozen LM with a linear head (Table 6). This is because topic classification largely depends on *which words appear*, and BoW captures this lexical signal directly. LM features are optimized for next-token prediction, not class separability, so a linear classifier struggles to extract topic information from them.

However, switching to an MLP head changes the picture. The Transformer + MLP is the only frozen setup that matches the BoW baseline, suggesting that Transformer features encode richer structure from self-attention that a nonlinear classifier can decode. Recurrent models benefit less from the MLP upgrade because their features, compressed through a sequential bottleneck, are already more "linearly shaped."

Across all models, lower perplexity correlates with higher downstream accuracy (Figure 6), confirming that better LMs do learn more useful representations. But the relationship is sublinear: the large PPL gap between LSTM and Transformer only translates to a modest accuracy gain with a linear probe. This suggests the extra patterns captured by stronger LMs are fine-grained and don't directly map onto coarse topic labels.

Fine-tuning the Transformer gives the biggest jump, reaching 93.1% accuracy (Table 6). Instead of using features shaped by "predict the next word," fine-tuning reshapes the entire representation space toward the classification objective. Validation accuracy climbs steadily over 10 epochs with no overfitting (Figure 8).

Looking at per-class results, Sports is the easiest class everywhere likely due to its distinctive vocabulary. Business is hardest because it shares terms with both World and Sci/Tech. Fine-tuning narrows this gap the most, as visible in the confusion matrix (Figure 7). The t-SNE plots (Figure 5) also confirm this visually: RNN features form overlapping blobs, LSTM features show moderate separation, and Transformer features have the clearest cluster boundaries.

Declaration

I declare that Claude was used in my assignment purely for syntax fixes and LaTex/markdown formatting. All experiments and analysis are original work.

Appendix: Figures

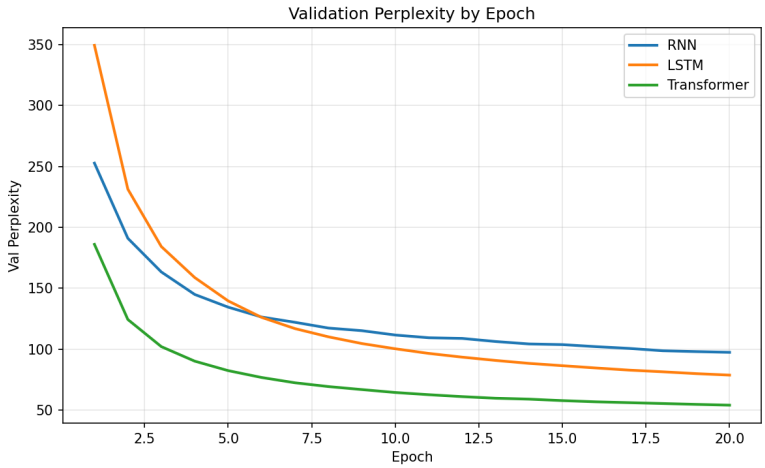

Figure 1: Validation perplexity over 20 epochs. Transformer converges fastest and reaches the lowest PPL.

Figure 2: Train vs validation loss. Small gap throughout indicates no significant overfitting.

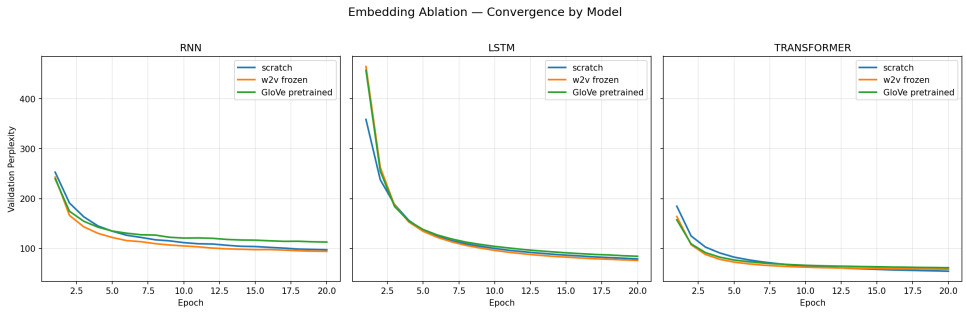

Figure 3: Validation perplexity for all 9 model-embedding combinations (Part II).

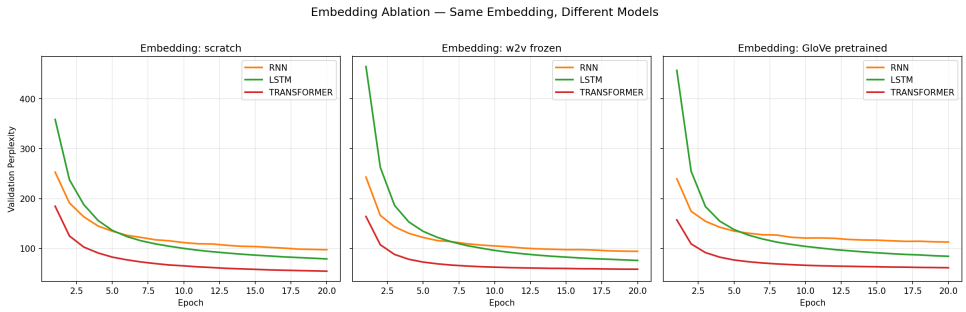

Figure 4: Same embedding, different models. Transformer wins regardless of embedding choice.

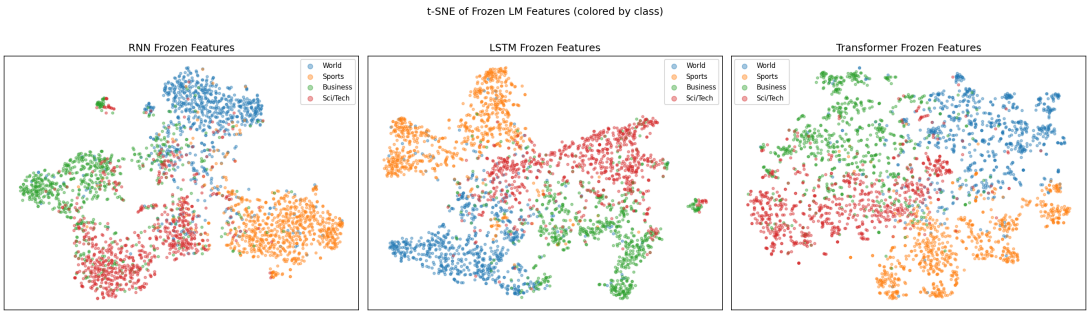

Figure 5: t-SNE of frozen LM features (3,000 test samples). Transformer features show the clearest clusters.

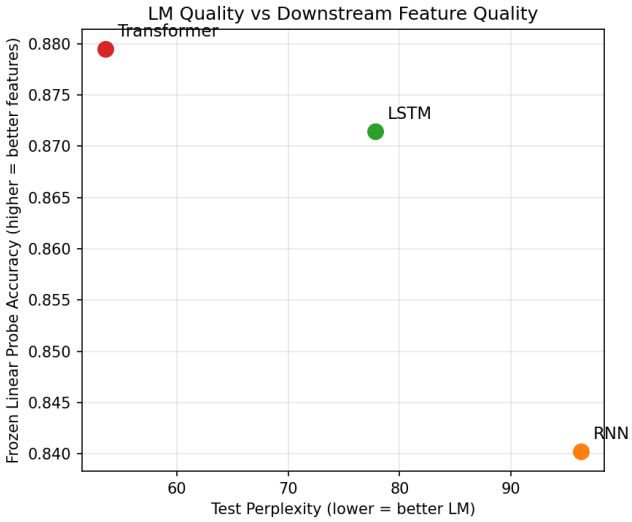

Figure 6: LM perplexity vs frozen linear probe accuracy. Lower PPL correlates with higher accuracy.

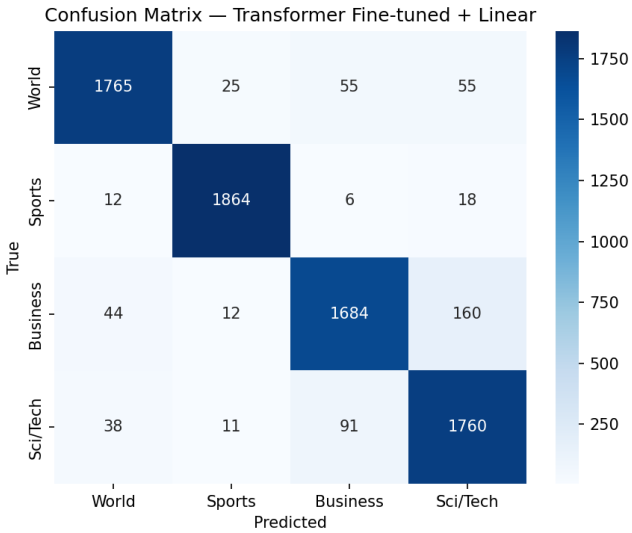

Figure 7: Confusion matrix for fine-tuned Transformer. Business-Sci/Tech is the main confusion pair.

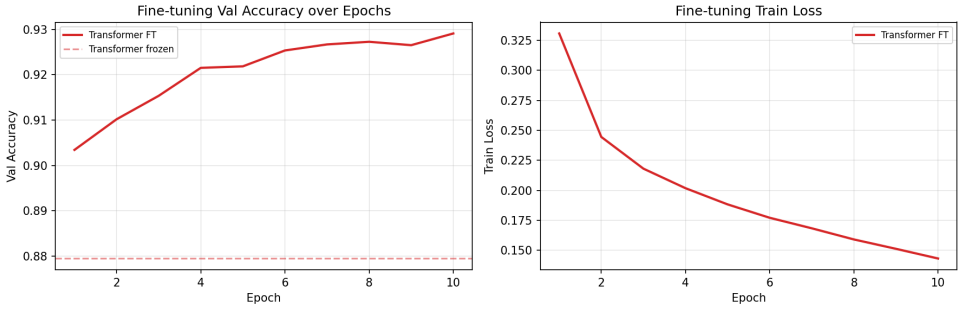

Figure 8: Fine-tuning dynamics. Val accuracy climbs steadily with no sign of overfitting.